IMPLEMENTATION DIRECTIVE / V1.0

Fable: Build the CAS computation path

Helm baseline repair. Structured representations.
Learned operations. Causal tests. Experimental integration.

The target

Make CAS structure participate in the computation, not just describe the answer afterward.

H A trustworthy, source-identified Helm baseline

X A standalone CAS Expression Lab with fixed reference semantics

I A tested, versioned integration into experimental Helm

Prepared for Little Henderson and Fable
21 September 2026

Includes the agreement/disagreement ledger, execution gates, evaluation design, and innovation mandate. The companion ZIP supplies 28 tasks, 52 planned acceptance vectors, seven role prompts, and editable schemas and templates.

Development handoff, not implementation acceptance. No Helm training, local checkout audit, cloud launch, or production deployment was performed in preparing this package.

Navigation

Read the execution decision first. Fable should use the editable directive and machine-readable work plan from the ZIP; this PDF is the review copy.

1. Start here: the execution decision 3

2. What is evidence, and what is a decision 3

3. Execution scope, resources, and adaptation 4

4. Team ownership and working rules 5

5. Q0: verify the checkout and close the evidence boundary 6

6. Q1: finish a real Helm baseline, not a stand-in demonstration 6

7. Q2: freeze a small operation contract 7

8. Q3: build behavior-paired SDG before training 8

9. Q4: implement representation and learned operation paths 9

10. Q5: prove use, not just readability 10

11. Evaluation design and proposed initial run settings 11

12. Q6: integrate into Helm without changing the claim by accident 12

13. Q7: PCBA extension after the language subset is real 12

14. Recovery, checkpoints, and run records 13

15. Delivery, stopping rules, and the first return 14

16. Source and interpretation notes 15

Agreements now explicit 16

Remaining disagreements and how to resolve them 17

Additional specifications, not established disagreements 19

What Fable is being asked to prove 20

Prepared for: Little Henderson and Fable
Date: 2026-09-21
Mission: Implement and run a bounded, source-grounded CAS computation pilot, then integrate the tested mechanism into an experimental Helm lineage.
Document status: Development directive and proposed experimental decisions. Not a new code audit, training result, spending approval, or production release.

> **Build a versioned CAS expression representation that preserves structure, is used by typed computational modules, can be decoded and inspected, and is tested for its actual contribution to the result. Do not stop at a registry, output labels, an attractive diagram, or a proposal for another proposal.**

1. Start here: the execution decision

Fable is the implementation lead. Use three coordinated workstreams rather than replacing Helm with the older CAS-Native transformer:

H: Restore confidence in the Helm 2.0 baseline. Locate the reported local branch and changes, close evaluator-integrity gaps, finish real optimizer and ladder wiring, and collect accurately identified baseline results.

X: Build a standalone CAS Expression Lab. Reuse audited compiler and engine semantics through a small adapter. Train and test an order-preserving expression encoder, decoder, and typed learned operations independently of both large-model integration and Helm's current pointer inputs.

I: Integrate the tested mechanism into an experimental Helm branch. Consume typed roles, bindings, and module results in the actual recurrent computation. Measure whether this changes performance and intervention behavior. Preserve the old pointer interface as a baseline.

This resolves the first-target question: **Helm is the intended beneficiary; the first new representation/module experiment is standalone; CAS-Native and the engine are sources of reusable semantics and utilities, not the new main project.** Only necessary library adapters should be extracted. Do not duplicate the full transformer or silently substitute its results for Helm's.

The first release must deliver a working vertical slice: a supported expression task enters, the learned path produces a typed result, a separate reference checks it, an intermediate binding can be inspected and deliberately corrected, and a fresh process reloads the exact model and reproduces the recorded behavior under the declared numerical policy. A failed learning hypothesis still yields a useful runnable experiment and a recorded negative result.

PCBA board support follows a frozen executable subset and parser. Jev, Dream-RSI policy improvement, full simplex geometry, and general language translation are not dependencies of this release. Keep their interfaces compatible, but do not let them expand the critical path.

2. What is evidence, and what is a decision

Use `sources/SOURCE_MAP.md` and `sources/SOURCE_MANIFEST.json`. The supplied reports are accounts of another agent's inspection. Fable must confirm the actual local code before treating paths, absences, parameter counts, or reported tests as current facts.

Source-supported starting points: the addendum accepts transition-consistency, intervention tests, explicit bindings, matched controls, and behavior-paired SDG. It reports executable CAS operations and separate CAS-Native components. The Helm handoff reports repaired state-dependent rounds and revised heads, but an unwired schedule and incomplete real-learning checks. It also reports a grader digest gap. [S1 §§Addendum, What the repo contains; S2 §§What is done, What is not done, Still open]

Reported local commits, not remotely verified here: `7f3a656f`, `9f61ffbb`, `c8d5c430`, `3459ea38`, and `27fd5698` on `research/helm-2.0-iterating-rounds`. The owner's pasted update says this branch was not pushed. Resolve each short SHA to a full local SHA. Do not recreate already-completed changes just because a remote view lacks them. [S5]

Results with limited attribution: `RESULT~1(6).JSO` identifies source `9b84b35c82bda422f744e76a170892313bc8b61b`, a reduced GPU schedule, eight main-training updates, and skipped checks. It does not, by itself, establish that the run used Helm 2.0. Retrieve its listed identity and loaded-source records before assigning the score to an architecture. [S4]

New decisions in this directive: the standalone-first strategy, quarantine of the three 28-way namespaces, the no-answer-leakage rule instead of an input ban, separation of reference and learned execution, source-grounded task splits, and staged acceptance. These are proposed implementation choices for this handoff, not retroactive consensus or alterations to historical evidence.

Use four separate statuses in every result: `engineering_status`, `representation_status`, `operation_status`, and `benefit_status`. A test suite passing does not set all four to PASS. Evidence of a type declaration is not evidence that the declared computation was learned.

3. Execution scope, resources, and adaptation

Proceed with reversible source inspection, isolated development, schemas, deterministic tests, package construction, and run preparation within the owner's existing policies. Execute neural smoke tests only on resources and within limits actually authorized for this task. The older proposal's phrase "$0" is not a new permission and does not mean local computation consumes no resources.

At intake, fill `templates/EXECUTION_AUTHORIZATION.json` from exact current instructions. Do not copy a historical dollar amount into a new allowance. Cloud launches, paid API calls, broad sweeps, production activation, destructive cleanup, and changes to protected acceptance rules need their applicable authorization. If local neural work is prohibited or its permission is unresolved, prepare the runnable experiment and ask one narrow resource question while continuing all unblocked work. Do not relabel CPU training as a deterministic test to bypass that restriction.

Innovation mandate: This plan is deliberately revisable. Unknown interfaces, incorrect source assumptions, unavailable resources, and negative experiments may require invention. Fable may change a mechanism when a better one is justified. Fable may not change the question, oracle, split, or pass mark after seeing confirmatory results and still call it the same experiment.

Use an adaptation record with the obstacle, evidence, options, chosen change, affected tasks/tests, cost, rollback, and reviewer. Routine reversible implementation details can be decided locally. Interface or experimental changes require a versioned contract and independent review before the affected run. Ontology changes, protected authority changes, spending, destructive actions, and production merges require the corresponding owner decision.

Limit unproductive retries: after two failed implementations of the same mechanism, diagnose the failure and record a revised hypothesis rather than repeating the same run. A controlled alternative encoder or decoder is permissible; silently adding a symbolic answer lookup to the learned arm is not.

If the standing push-after-commit instruction applies to this checkout, obey it for the designated authorized remote branch after tests and secret checks. Design opinions or an experimental architecture do not by themselves justify ignoring that instruction. Do not force-push, delete branches, publish protected test data, or merge to production. If instructions or remote ownership conflict, record the conflict, preserve commits, and ask only about that conflict.

4. Team ownership and working rules

Role	Owns	Must not do
F0 Fable, lead integrator	Discovery, task assignment, dependency closure, adaptation, integration, release	Approve its own confirmatory learning claim
F1 Helm engineer	Baseline repairs, worker wiring, matching output contract, reload/recovery	Change frozen answers or historical evidence
F2 CAS semantics engineer	Operation contracts, engine adapter, 40-coordinate conversion, executable subset	Invent semantics for syntax-only components
F3 Data engineer	Behavior-paired SDG, templates, splits, leakage tests	Expose sealed final targets to the learner
F4 Representation engineer	Encoders, decoders, modules, interventions, three model arms	Call the reference solver inside the neural prediction path
F5 Evaluation custodian	Independent oracle, final split custody, trusted scoring, acceptance	Tune models on final evaluation outcomes
F6 Adversarial reviewer	Corruption tests, bypass tests, reproduction, evidence audit	Fix production code during an independent audit run

A smaller agent team may combine development roles. It must not collapse final-evaluation custody into the same unrestricted session that designs the model. If independent access controls are unavailable, report `EVALUATION_NOT_INDEPENDENT` and do not promote a protected generalization claim.

Use isolated worktrees and ownership of files. Do not edit shared code during another agent's frozen run. Discover `.claude/worktrees/agent-*` and other stale-looking clones using metadata; exclude them from primary searches until explicitly selected. Do not delete them as a convenience. They may contain unmerged user work.

Each handoff includes full commit SHA, source hash, files changed, exact command, environment identity, measured outcome, failures, and next dependency. Every work item in `TASKS.json` has one accountable owner and an independent reviewer. New helpers and schemas are new APIs until implemented and tested; do not describe them as existing Helm functions.

5. Q0: verify the checkout and close the evidence boundary

Discover the root from the actual working directory, not a guessed Windows path. Read active repository guidance, `BRAIN_LINEAGE.md`, the independent audit, resource rules, and successors to the attached reviews. Use targeted paths and Git-tracked search before recursive filesystem searches. Inventory branch heads, dirty changes, active worktrees, running jobs, protected artifacts, and import resolution.

Run the supplied structural checks if present, under their documented environment. Locate their successors if they were renamed. Reproduce the state-message difference and controlled two-hop perturbation. These test information propagation, not complete reasoning competence. Test whole-model predictions separately. A nonzero hidden-state change alone is not a learning result.

Repair the grader boundary before any claim-bearing run. The trusted scorer identity, rubric, target-set identity, and input/output schema must be anchored outside the run directory the learner can modify. Verify the hashes of predictions, scoring files, and listed artifacts; then independently rescore the raw predictions. Do not accept run-supplied totals as authoritative.

Reject: an altered score file; an altered score file plus rewritten run manifest; an altered evaluator plus a matching run-owned digest; a wrong source profile; duplicate question IDs; missing/extra predictions; a result for a different split; unsafe paths; malformed or nonfinite numbers. A hash proves byte agreement with a reference, not who established that reference.

Preserve the original frozen 18 questions, pass mark, and rehearsal copy. Add a versioned audit supplement recording the reported 11/18 zero-learning result and same-topology issue with exact provenance. Do not silently rewrite the original experiment. If historical artifacts are sufficient, independently rescore them; otherwise mark the limitation explicitly.

Q0 exit: resolved source and import identities, protected scorer anchor, passing tamper tests, preserved user work, and a claim ledger distinguishing reported, reproduced, contradicted, and unresolved statements. Semantics/data work may proceed while Q0 is repaired, but no new learning claim may bypass Q0.

6. Q1: finish a real Helm baseline, not a stand-in demonstration

Confirm whether the latest local code already implements the reported F1/F2/F4 repairs. Wire `planned_main_updates` from the actual schedule plan, not a copied literal such as 1470. Test that the real learner takes optimizer steps and records the learning rate, clipping settings, pre/post gradient norm, update index, and main-versus-replay step class. A stand-in preflight cannot satisfy this requirement. [S2 §§Learning-rate seam, Verification state]

Repeated validation loss belongs to a development panel, not the protected final test. Keep the original frozen gate untouched and report a separate difficulty ladder by world size, hop distance, question kind, and topology family. The handoff reports 1,072 ladder questions and a nonzero simple-baseline envelope; confirm those records rather than treating chance as the baseline. [S2 §F3]

Audit the correspondence task contract. Determine whether it requires a complete bijection, an injective partial match, or unmatched elements. A soft Sinkhorn matrix does not enforce a valid discrete assignment after independent row argmax. Add task-appropriate joint decoding and validation, with deterministic tie handling. Report pre-decoding scores and post-decoding exact

accuracy separately; decoding is a real computation, not free learned competence. [E3; derived counterexample in AGREEMENTS_AND_DIFFERENCES.md]

Keep recurrence, reach-head, matching, and optimization contributions distinguishable. Start with the repaired baseline; use bounded matched ablations where authorized. More rounds can expand message reach but may not improve answers. Width, additional attention, and larger training runs are follow-on experiments, not promised gains. No claim that one loop defect explains every failure is required for this plan.

Use strict checkpoint identities. A Helm 1.0 checkpoint can be evaluated under its own code, or explicitly migrated into a new experimental model with a complete tensor report. No silent dropped tensors or fresh initialization may be described as exact restoration. For exact training resume include optimizer state, schedule position, data cursor, RNG state, and environment identity.

Q1 exit: reproducible real-worker smoke within permission, correct schedule accounting, complete matching output checks, source-bound checkpoints, and same-question baseline results. If permitted compute is missing, report `ENGINEERING_READY / EXECUTION_BLOCKED_RESOURCE`, not PASS.

7. Q2: freeze a small operation contract

Create `OPERATION_CONTRACTS.json` before training. Reported candidates are `compose`, `involute`, `is_admissible`, domain descent, and the RPM/prerequisite gate. Existing code must be read for each function's actual signature, domains, effects, and numerical behavior. [S1 Addendum]

An operation is admitted only when it has a versioned identifier, authoritative definition, pinned implementation, input/output types, valid domain, invalid-input behavior, mutation policy, numerical policy, and independent expected-output cases. Conflicting definitions get a discrepancy record. Executable code is not automatically a correct implementation of every prose claim with the same name.

Use operation-specific types:

```
D_k(M_k(E_k(x))) = T_k(x)
```

The equality is exact for declared symbolic or Boolean results, structural for typed records, and tolerance-based only where the contract explicitly requires continuous quantities. `UNKNOWN`, `INVALID_INPUT`, `UNSUPPORTED_OPERATION`, `NUMERICAL_FAILURE`, and `BUDGET_EXHAUSTED` are not interchangeable.

Treat `RegisterFiLM` and similar parameterized components carefully: they may be trainable mechanisms rather than fixed symbolic oracles. A registry such as `OmegaRegistry` is not automatically an operation to learn. Do not fabricate one module per filename. Start with at least two genuinely distinct supported operations; add a directional or context-sensitive operation only after its semantics are confirmed.

Reference/learner separation: the fixed engine generates targets and checks results in the evaluator process. The learned arm receives permitted inputs and operation IDs, not reference outputs, target-derived masks, an evaluator handle, or symbolic solver callbacks. A production hybrid arm may invoke exact operations, but must be labeled `HYBRID_FIXED_OPERATOR`, not neural acquisition.

Adopt a provisional lab-local coordinate convention: domain-major `index = 10*domain + noetic`, with explicit mappings for all 40 states to tokenizer IDs. Verify the actual labels before applying it. This

adapter does not renumber the source engine or migrate historical data. Keep legacy A1-D10, Noetic/domain literals, and numeric IDs explicitly related, not implicitly identical.

Keep three 28-way namespaces separate: Drive × Domain, Drive × Register, and the TKS Foundation/World-aspect scheme. Retain their original metadata but do not train a unified 28-way head in this pilot. A missing owner choice for a unification therefore does not block the 40-Element expression work.

Q2 exit: independently tested operations, explicit coordinate maps, typed result contracts, and quarantined ambiguities. Count unsupported CCDL leaders as the reported 17/39 until re-audited; do not repeat the source's inconsistent 'two-thirds' paraphrase.

8. Q3: build behavior-paired SDG before training

Extend the existing generator through an isolated, versioned package or adapter. Preserve old outputs and frozen schemas. The reported template-first direction is CAS → English; it can supply input-target pairs for a separately trained inverse interpreter. It is not already that interpreter. [S1 §The SDG runs the other way]

Each new record must bind: source/ontology versions, task family, raw or structured observation, canonical input AST, role and entity bindings, operation ID, context/prerequisites where used, oracle result, independently checked behavior trace, validity status, invalid alternatives, split/group keys, generator seed/version, and evidence provenance. Store targets and oracle traces outside the model-visible packet.

Implement explicit visibility manifests for three task modes:

Mode	Model sees	Withheld
Reconstruction	An expression encoding input	Target canonical AST during free-running decode
Neural operation execution	Operand representation and requested operation	Reference result, answer-derived masks and traces
Native interpretation, later	Generated language or other observation	Target CAS expression and reference answer

Supervised targets are allowed for computing training loss. They are not allowed as model inputs during evaluated prediction. A teacher-forced training pass is not an autoregressive reconstruction evaluation. Record both paths separately.

Use same-word/different-binding pairs, same-structure/different-wording pairs, repeated types in different roles, invalid and unsupported cases, longer expressions, and mixed-operation chains supported by the frozen type system. Negatives must be checked against the relevant contract, not merely labeled by a language model.

Avoid foreign-model answer targets in the native lane. Deterministic templates are the default. Any separately authorized foreign paraphrase must preserve an independently established source expression and receive its own provenance and semantic checks; it never defines CAS semantics. Ambiguous inverse examples need a set-valued interpretation or an ambiguity outcome, not a forced

unique answer.

Split by canonical AST or graph family before generating paraphrases. Keep siblings, renamed variants, near duplicates, equivalent derivations where relevant, and shared original board instances in the same split. Report exposure at different levels: surface form, AST shape, operation sequence, graph isomorphism class, and semantic result. Familiar atomic operations inside new compositions are not themselves forbidden; disclose that exposure.

Do not use source hashes, line numbers, filenames, split tags, case labels, chronology, or IDs as covert answer features. Hashes are evidence identifiers, not neural inputs by default. Run substring, type-equality, size/degree, template, nearest-neighbor, and appropriate exact symbolic baselines before admitting a task as reasoning evidence.

Q3 exit: reproducible generation, sealed visibility boundaries, independent labels, non-overlapping declared groups, and a baseline report explaining exactly what each task tests.

9. Q4: implement representation and learned operation paths

Build under a proposed namespace such as `research/helm_cas_expression_lab/`; adopt an existing equivalent if discovery finds one. Do not write a second package merely to match this name.

Representation R0: token-row sum plus nearest-neighbor, as a diagnostic control. It is expected to discard order, and that limitation must be visible.

Representation R1: an order-preserving sequence or tree encoder with a grammar-aware decoder. Widths 64 and 128 may be used as initial engineering settings if allowed by measured resources. They are not theoretical capacity limits.

Representation R2: variable-sized AST or role-graph memory with learned vectors attached to nodes, typed operator slots, explicit bindings, context, and versioned identity. R2 is the preferred path toward Helm integration if its tests justify it. A $40\times k$ concept feature may be explored within it, but is neither mandatory nor proof of semantic computation.

Separate compression tests from structured-memory tests. For strict bottleneck reconstruction, the decoder must receive only the declared bottleneck and permitted generation history. No original AST, raw input cross-attention, pointer-copy channel, or exact-binding side channel can be silently retained. For R2, such structure may be deliberately available, but then report structured computation rather than lossless compression into a single vector.

Canonical AST equality must preserve task-relevant operand order, parent/child position, and bindings. Do not simplify away those distinctions merely to inflate round-trip scores. Report parse validity, canonical exact match, role/binding accuracy, rejection accuracy, and semantic execution accuracy separately. Valid syntax is not correct semantics. [E2]

Train modules against the Q2 contracts. Fixed label dictionaries and routing on an operation ID in the problem are allowed. Fixed computation of the answer is not allowed in the learned-execution arm. Add runtime import/call instrumentation and sandbox restrictions to demonstrate this separation.

Watch for trivial algebra: the reported `(a+b) mod 10` operation is commutative and associative. A modular-sum algorithm can solve nested repetitions without preserving grouping. Use it as a strong baseline and a narrow conformance test, not evidence of general order-sensitive reasoning. Choose a

confirmed asymmetric/context-sensitive task for that separate claim.

Track the loss components for reconstruction, typed outputs, operation execution, and interventions. Tune their weights on development data only. Do not force every scalar into a CAS category, or infer actual physical current, consciousness, full-board behavior, or universal meaning from a good loss curve.

Q4 exit: free-running reconstruction, isolated learned modules, correct invalid-case handling, and an executable end-to-end experiment. An encoder/decoder alone does not complete the mission.

10. Q5: prove use, not just readability

Run at least these three neural arms on the same permitted information, data, and task definition:

Arm	Mechanism	Purpose
A Conventional	Baseline network without CAS intermediate supervision	Measures ordinary learned performance
B CAS heads only	Same task path with CAS prediction/decoder heads	Tests whether labels improve or merely accompany prediction
C CAS computation	CAS-structured intermediate state used by typed learned modules	Tests the proposed internal mechanism

A and B must not receive less task information just to help C. C must not receive true intermediate concepts at test time when A and B receive raw observations. Use a separate oracle-concept ceiling to quantify interpreter error. Report all active and auxiliary parameters, training FLOPs or honest proxies, wall time, data exposure, and tuning budgets. Approximate capacity matching should not use inactive parameters as fake equality.

Include exact symbolic execution as a usefulness baseline. The experiment is not required to outperform an exact solver on its own deterministic task. Distinguish neural fidelity, structural transfer, causal use, and end-to-end operational benefit. Benefits could come from interpretation, reuse, or diagnosis, but must be measured at the relevant task boundary.

Intervention protocol: select task pairs with a known input-binding or context difference and independently computed target behavior. Freeze the model. Record the original prediction; inject a controlled intermediate error; replace only the selected concept/binding with its correct value; continue through the same learned downstream modules. Do not feed the final answer, choose patches by their success on the test answer, or edit hidden state unrelated to the declared intervention. [E1]

Report artificial-corruption repair separately from corrections of naturally occurring errors. A successful repair of an injected mistake demonstrates controllability in that setting, not a general debugging ability. Report every eligible case, not just successful repairs.

Add sham interventions, semantically irrelevant edits, wrong concept patches, typed-role permutations that actually change the reference outcome, module replacements, and CAS-head-only interventions. Verify positive controls first: a meaningful perturbation must be capable of changing the reference behavior. A global consistent relabeling of a permutation-equivariant computation may

correctly have no effect and is not a falsification of semantic use.

A strict-bottleneck C variant must not bypass the CAS intermediate state. If a residual path is added, disclose it and run path ablations; the stronger no-bypass claim no longer applies. Inspect whether tiny variations in purported concept probabilities carry answer information. Decodable concepts are necessary for the proposed interface, not sufficient for faithful explanations.

Q5 exit: separately reported reconstruction, operation, intervention, and transfer results; frozen/adaptive and trained/untrained comparisons as applicable; meaningful controls; and no concealed reference-solver path.

11. Evaluation design and proposed initial run settings

The numeric settings in `templates/EXPERIMENT_PLAN.json` are suggested starting settings, not reported results or universal scientific thresholds. Fable and the evaluation custodian must freeze or revise them before confirmatory runs, using development evidence and actual permissions. No post-test adjustment can retain the same experiment identity.

For a no-new-spend engineering smoke, the template starts with one seed, at most 128 optimizer updates per model, and at most 2,048 generated training records, 256 development records, and 256 engineering-evaluation records. It proposes 64-dimensional representations first and a bounded 128-dimensional alternative. These are ceilings, not a request to generate redundant records when a domain is smaller. Neural execution remains permission-gated. No confirmatory capability claim is licensed by this smoke.

The confirmatory template uses at least three independent initialization/data-order seeds and at least two semantically distinct task families, subject to an explicitly approved budget. Structural holdouts and longer inputs should be evaluated separately from fresh coefficients or renamed IDs. Keep final targets and diagnostic outputs under the evaluator's control. All developers who saw earlier benchmark examples must treat those examples as known engineering fixtures, not protected holdouts.

Default proposed acceptance criteria are deliberately separate:

Engineering: all applicable mandatory integrity, visibility, schema, reload, and no-unauthorized-action tests pass. Any required skipped check leaves the affected gate INCOMPLETE.

Representation/operation pilot: at least 95% free-running canonical or typed-result exact match within each preregistered supported evaluation stratum, with invalid-input handling reported separately. This is an engineering target, not proof of a formal language's complete coverage. Product output still requires the unchanged native checker.

Causal-use pilot: at least 90% expected restoration on a preregistered eligible injected-binding cohort, at least 98% decoded-outcome invariance on declared sham/irrelevant edits, and a contrast with a heads-only path. These are proposed targets; natural-error repair remains a separate metric. Failure to meet them must be reported, not hidden by cohort selection.

Benefit: no automatic percentage guarantee. Before final evaluation, freeze one primary benefit endpoint and a smallest worthwhile effect using development results. A comparative claim requires a paired estimate against the strongest applicable matched baseline, uncertainty that respects shared worlds/templates and seed variation, and no failure of integrity or correctness gates. Choose sample

size from that design rather than declaring thousands of near-duplicate questions independent.

Report no-learning performance on identical items. A difference between 11/18 on one panel and 45/127 on another has no interpretable lift. Do not claim new topology from new names. More exact questions on an exposed set do not create independent validation.

12. Q6: integrate into Helm without changing the claim by accident

After Q1 and Q5, adapt the accepted representation/modules to a separately versioned Helm candidate, provisionally `helm-cas-experimental/0.1`. Final lineage labels follow the actual repository policy. Do not rename it Helm 3.0 or promote it over Helm 2.0 solely because this plan discusses a typed input door.

A proposed integration packet carries node identities, typed edges, operand roles, exact input bindings, operation IDs, and learned feature vectors. Typed relations must influence recurrent messages or module selection in the computation, not appear only in a log after prediction. Keep relation order/direction where meaningful. Preserve a legacy-pointer adapter for baseline experiments.

Distinguish three pathways in traces: learned prediction, fixed symbolic calculation, and native verification/commitment. If hybrid exact operations are useful, keep them; mark their contribution honestly. An evidence checker returning a corrected final answer must never be counted as the learned model's original answer.

Do not make the 40-coordinate summary the full state identity. Maintain exact object, expression, context, evidence, and model IDs. Optional profile values are descriptive and remain separate from confidence and native work counters.

Add a bounded read-only CAS-RPA export for the experimental path, using a new adapter version. It should expose input expression, bindings, selected operation/module, pre/post intervention, learned result, reference result, verification outcome, and model identity. Do not reinterpret a new journal as runtime 0.2. Navigation changes focus only. Visual proximity and overlap never create execution evidence.

Release demonstration: load a fresh task, show the typed representation, make a learned prediction, show the independent check, inspect a controlled intermediate error, correct only the binding, recompute, and reload the saved model in a separate process. Also show an unsupported case. A scripted fixture must be labeled as such; it is not the whole benefit study.

Q6 exit: actual local computation through the experimental model, reproducible report and optional atlas projection, baseline compatibility, explicit lineage, and a rollback path. A registered parameter name alone does not satisfy the integration.

13. Q7: PCBA extension after the language subset is real

The reported CCDL source contains typed cause/bias/effect ports, multiple edge forms, parameter blocks, feedback, and unresolved statements. The reported example has loose connections. These are reasons to define a smaller executable subset, not permission for a parser to choose the answer key. [S1 §A CCDL board]

Freeze the subset and create an explicit ambiguity ledger. Parse only declared netlist statements. Reject or exclude unresolved edges and syntax-only constructs. Corrections to book examples require a separately versioned authored fixture, not an invisible parser inference. Known book boards are regression/integration cases because developers have already seen them; generate fresh held-out board families for protected evaluation.

Represent component instances, net/state nodes, typed ports, relation kinds, and multi-input dependencies explicitly. A bipartite or incidence-graph representation may encode hyperedges without pretending every relation is a simple untyped adjacency. Signal feedback is not a containment cycle: preserve acyclic `contains_part` separately from cyclic signal/control relationships.

Begin with static typed structure and exact finite behaviors the subset actually defines. Do not invent continuous physical dynamics for a filter, fuse, oscillator, or amplifier from its name. Add such behavior only when the component contract specifies state, units/scales, update rules, and numerical checks.

Separate direct-read tests such as identifying a declared bias port from multihop reasoning and constructive modification. Strong typed-edge lookup is an appropriate baseline for a port query; it need not score at chance. Reasoning tests must require a genuine additional operation that the tested baseline lacks, with matched input information. Numeric properties require an explicit numerical input channel and typed units/scales.

Extend SDG with this subset only after the parser and independent evaluator pass. Train on newly generated board structures, hold out families and transformations, and test changed parameter values and relation types without leaking expected outcomes. Benchmark learned and exact symbolic paths separately.

Q7 exit: a versioned parser, executable subset, generated data, held-out board evaluation, and typed recurrent integration. If semantics are unavailable, ship the earlier CAS computation release and mark the board lane BLOCKED_SEMANTICS. Do not call the whole project blocked.

14. Recovery, checkpoints, and run records

Freeze the source tree before every run. Record full commit SHA plus dirty diff hash if a dirty tree is explicitly permitted; otherwise require a clean run worktree. Record the imported files, dependency versions, architecture schema, tokenizer/ontology versions, dataset and evaluator identities, and device/numeric policy. Do not edit run sources in place.

Create output paths that do not already exist when the runner requires that. Use an authorized durable terminal/job runner for long work rather than assuming an agent-tool timeout equals model failure. Never kill unrelated jobs. Budget reservations, elapsed time, and interrupted phases remain in the ledger.

Save model tensors, optimizer state, schedule state, RNG states, data cursor, run config, and update provenance. Test resume at each durable boundary in a bounded representative run: before first update, after an update, after checkpoint write, before evaluation, after evaluation, and during release assembly. Test incomplete checkpoint writes and mismatched manifests.

Exact bitwise replay is required only where the pinned environment and operation settings support the declared policy. Otherwise define numerical tolerances before the run and require identical

decoded semantic outcomes where the task requires them. Never erase real nondeterminism by widening tolerances after seeing a discrepancy.

List neural and replay updates separately. If an old result lists artifacts but their bytes are unavailable, record that only the manifest was inspected. Do not authenticate a checkpoint from its filename. Code restoration, lineage migration, training resumption, and a fresh experiment are different operations.

Record failures as typed outcomes and retain their artifacts. An interrupted or skipped phase remains INCOMPLETE. A clean JSON file and `status: COMPLETE` are not enough when required checks are skipped. Report failed candidate models without overwriting the best earlier evidence.

15. Delivery, stopping rules, and the first return

Fable must deliver implementation commits, an exact runbook, versioned operation/data/architecture contracts, model and data manifests, independent evaluation records, and an agreement/decision ledger. `TASKS.json` and `ACCEPTANCE_TESTS.json` are the execution checklist; update status in a separate progress file, not by rewriting the frozen plan during a run.

The first return should contain: resolved branch/HEAD; a map of the five reported commits; current protections and permissions; existing code to reuse; open evidence defects; tasks assigned; and the next actual command. It must include completed code/test progress where possible, not only another roadmap. Do not ask the owner to choose the 28-way ontology, model family, every width, and every file path before starting independent work.

A release report must state:

- Which paths really executed and which remain proposed.
- Which operations and expression lengths/structures are supported.
- Reconstruction, operation, intervention, and transfer scores separately.
- Same-item baseline comparisons and failed/non-beneficial findings.
- Exact source, model, data, evaluator, and resource identities.
- Whether the independent evaluator and all mandatory checks completed.
- Any local-only commits, push status, blocked resources, and unresolved semantics.

Stop a specific experiment for data leakage, scorer compromise, wrong identity, resource-limit violation, or unresolved oracle disagreement. Preserve the evidence and repair the cause. Do not stop the entire program because an optional external dependency is absent. If repeated clean tests show no CAS advantage, retain the working symbolic/hybrid tooling and report `NO_MEASURED_BENEFIT`; do not manufacture a new favorable benchmark after the fact.

Definition of done: at least one complete, independently checkable CAS representation-to-operation-to-intervention path is implemented and executed under permitted resources; the experimental Helm integration is either evaluated or has an exact, bounded blocker; the source-derived claims and measured benefits are not conflated. Full PCBA, language breadth, and unrestricted self-improvement remain separate milestones.

16. Source and interpretation notes

[S1] `HELM_CAS_EXPRESSION_SPACE_REVIEW(1).md`, including its addendum, supplied by the user. Source for reported SDG direction, namespaces, Helm input interfaces, component gaps, accepted consistency/intervention proposals, and remaining disagreements.

[S2] `HELM_2_0_HANDOFF.md`, supplied by the user. Source for reported branch, fixes, unfinished worker/evaluation paths, checkpoint risks, local-rule references, and grader issue.

[S3] `HELM_ARCHITECTURE_FINDING_AND_PLAN.md`, supplied by the user. Source for the reported frozen-message defect, code excerpt, measurements, and proposed F1-F5 plan. This directive narrows several causal and scaling claims rather than silently adopting them.

[S4] `RESULT~1(6).JSO`, supplied by the user. A worker result/manifest with reduced-run outcomes and attribution limits, not the complete evidence bundle.

[S5] `sources/CLAUDE_OWNER_UPDATE_EXCERPT.md`. A faithfully labeled excerpt of the owner's pasted update, including the local-only commit table. It is not a Git verification receipt.

[E1] Koh et al., *Concept Bottleneck Models*, ICML 2020, PMLR 119. Published precedent for concept-mediated prediction and interventions; not CAS-specific evidence.

[E2] Kusner et al., *Grammar Variational Autoencoder*, ICML 2017, PMLR 70. Published precedent for encoding/decoding grammar trees; syntax validity does not establish semantic correctness.

[E3] Mena et al., *Learning Latent Permutations with Gumbel-Sinkhorn Networks*, 2018, arXiv:1802.08665. Continuous relaxation of matching; not a discrete assignment guarantee.

Official source URLs and exact attachment hashes are recorded in the package source map and manifest. Reference pages were checked during preparation. The current local checkout was not inspected and no Helm/CAS training was run in preparing this directive.

Appendix: where the reviews agree and differ

Prepared: 2026-09-21. **Status:** source-grounded comparison plus explicit engineering decisions; not an independent checkout audit. Source identifiers refer to `sources/SOURCE_MAP.md`.

The latest Claude addendum changes its original rejection of component-matching. It accepts a behavioral test against an executable CAS operation, a controlled intermediate intervention, matched comparison arms, explicit bindings, and behavior-paired synthetic data. These are genuine areas of agreement. Do not continue quoting the earlier NOT LICENSED paragraph as though the addendum did not revise it. [S1 addendum]

The original output-only prohibition and several causal interpretations remain disputed. This ledger does not claim Claude accepted corrections introduced after its pasted response.

Agreements now explicit

ID	Shared position	Implementation consequence
A01	Elements organize content; component contracts organize transformations.	Maintain typed expression content separately from parameterized operators.
A02	Individual floating-point weights need not have a unique readable CAS meaning.	Attribute behavior to a declared module and its tested contract, not to a name on a scalar.
A03	CAS-Native has reusable components, but those components are not already integrated into Helm.	Reuse audited resources through explicit adapters; keep model/run identities distinct.
A04	A learned operation can be scored against a specified CAS reference.	Implement operation-specific consistency and error-domain tests.
A05	A decoder or label alone is not enough for the stronger claim. Claude adopts the intervention test.	Correct an intermediate binding without supplying the answer, and evaluate downstream behavior.
A06	Exact roles and bindings matter; a profile does not preserve everything.	Preserve operand roles, order, relations, context, and the original problem identity.
A07	Compare conventional, CAS-heads-only, and CAS-computation arms.	Match information, capacity, data exposure, and tuning budgets; report residual differences.
A08	Behavior-paired SDG is useful.	Record inputs, operations, prerequisites, outcomes, and negative cases with independent checks.
A09	Syntax-only PCBA constructs are not executable semantics.	Start with a verified subset; introduce boards after parser and contracts are frozen.
A10	A shuffle control can be vacuous; ontologies must be versioned.	Test that a semantic corruption can affect the reference, and keep all three 28-way namespaces separate.

A01-A10 are grounded in the latest addendum and surrounding review. They do not establish that any new experiment passed. [S1 lines 270-343 in the supplied review]

Remaining disagreements and how to resolve them

D01. CAS input versus leaked answers

Claude's position: CAS belongs in the output and registry, never in inputs; existing Helm analogy can exploit type equality. The review identifies a real substring-label example but labels the universal compiler-ceiling claim as argued, not run. [S1 §§The one-paragraph answer; What the evidence kills; Claim states]

My position: the shortcut invalidates that particular task as evidence of the intended skill, not every structured input design. A problem expressed in CAS is not necessarily its answer. A native predicted CAS intermediate representation is another legitimate experimental design.

Decision: preserve the existing frozen input policy. In a new versioned experimental task, permit answer-free typed descriptions. Prohibit target labels, oracle-derived outcomes, and hidden solution methods from the model-visible packet. Compare against type-equality, substring, and exact symbolic baselines. Do not make CAS input a performance claim before testing it.

D02. The SDG runs in the other direction

Agreement: the documented generator starts with CAS structure and emits English; it is not a native general inverse translator. [S1 §§The SDG runs the other way]

My refinement: paired CAS-to-English examples can train a separate English-to-CAS learner. The target is allowed in supervised training, not as an inference input for the same prediction. That does not turn a foreign model's proposed interpretation into ground truth.

Decision: use deterministic/source-checked generation first. Put gold expressions and intermediate answers in a separate target channel. Keep natural-language interpretation outside the first algebra-only critical path. Add it later as a distinct experiment with ambiguity handling and unseen templates.

D03. Vector closure and expression validity

Claude's concern: arbitrary vector sums/scalings remain vectors, whereas some CAS expressions are invalid. [S1 §What the evidence kills, item 1]

My position: an encoder can map legal expressions into a subset of a vector space, and a decoder can reject other points. The subset need not be closed under arbitrary arithmetic. This does not imply that a finite-precision fixed vector can losslessly store arbitrary-length expressions.

Decision: compare a bounded strict bottleneck with an order-preserving sequence/tree representation. Variable-sized structured memory is the preferred integration option. Keep legal syntax, semantic validity, and latent geometry separate. Published grammar-latent work is a precedent, not validation of CAS. [E2]

D04. Fixed algebra versus learned modules

Claude's combined sequence: make algebra fixed inside the model, then train modules to match the executable operations. [S1 §§Fix, never learn; combined plan]

My refinement: those are two different execution paths. A learned candidate cannot obtain its answer by calling the fixed reference and then receive credit for learning.

Decision: freeze the reference interpreter outside neural prediction. Train the candidate against its labels. Keep a separate, useful fixed-symbolic/hybrid product arm and label its contribution. Discover whether RegisterFiLM has trainable parameters; a parameterized transform is not a universally fixed algebraic oracle. A registry alone is not an operation.

D05. Which project receives the first experiment?

Claude's suggestion: start where the executable semantics already live, with the owner choosing CAS-Native or Helm.

Decision in this directive: first build a standalone CAS Expression Lab using audited engine/CAS-Native resources. Finish the real Helm baseline in parallel. Helm remains the intended integration target. This avoids choosing between a stale transformer integration and an unready pointer interface. It is an implementation choice for the handoff, not a claim that the repositories are already connected.

D06. Dataset ordering

Claude's combined sequence: consistency/intervention work precedes behavior-paired SDG. [S1 combined plan]

My position: define the task, oracle, behavior records, and held-out splits before training and intervention evaluation. Otherwise the experiment can drift toward what the candidate happened to learn.

Decision: contracts and behavior-paired data precede module training. Add more generation features later as versioned extensions, not after inspecting the final evaluation.

D07. Recurrence is a defect, not a complete causal diagnosis

Agreement: the report identifies neighborhood messages computed from frozen input instead of updated node state. Fixing that is an important mechanism repair. [S3 §1]

Disagreement: identical messages do not imply every hidden state is identical. The reported recurrence still changes its local state. Global pooling and heads also exist; the local two-hop test is not a proof that every whole-model output has only one-hop information. A falling gradient does not prove that all further training is useless. [S3 §§1-3]

Decision: verify the local repair, then measure whole-model behavior using controlled ablations. Do not require agreement with a sole-cause explanation. More rounds, wider networks, and schedules remain hypotheses, not guaranteed gains.

D08. Sinkhorn is not a discrete assignment guarantee

Claude's original framing: a few normalizations make one-to-one correspondence an output property. [S3 F2; S2 matching details]

My refinement: Sinkhorn is a continuous matching relaxation. Even an exactly doubly stochastic matrix can produce repeated target selections under independent row argmax. [E3]

For example, rows of [[0.40,0.35,0.25], [0.45,0.30,0.25], [0.15,0.35,0.50]] and columns all sum to one, but the first two row maxima select the same column. This is direct arithmetic, not a Helm result.

Decision: specify the actual task's complete/partial matching rules and jointly decode or validate accordingly. Independent pair scores can represent a correct set of preferences; they simply do not enforce the joint constraints by themselves. Report soft scores and decoded assignments separately.

D09. Baselines need not be at chance for useful tasks

Claude's proposal: require a one-hop/literal baseline at chance on new board questions. [S1 §PCBA boards]

My position: direct port lookup should be easy for an exact typed parser. That can validate an interface without proving advanced reasoning. A baseline above chance does not automatically mean leakage; it can mean the task contains usable information.

Decision: separate basic contract tests, genuinely compositional tasks, and construction tasks. Measure lift on identical appropriate tasks. Keep strong exact baselines rather than hiding input information to force neural necessity.

D10. A registry and reconstruction test are useful but incomplete

Claude: proposes a compiled registry, decoder, and held-out round-trip. The addendum also accepts causal intervention. [S1 §§1-2 and addendum]

My refinement: a declared operation name does not certify its weights; reconstructing an expression does not establish that it governed the answer. Gold tree side channels can make decoding trivial.

Decision: report reconstruction, typed operation fidelity, causal participation, and comparative benefit separately. Instrument the prediction path for oracle calls and bypasses. A permitted full structured memory must not be advertised as strict fixed-vector compression.

Additional specifications, not established disagreements

Forty coordinates: differing orderings can be connected by a tested permutation when meanings agree. Use domain-major internally in the isolated lab, preserve tokenizer IDs and round-trip tests, and do not change the global ontology. [S1]

Twenty-eight categories: preserve three explicit namespaces and defer a unified 28-way learned head. The user's compositional-40 clarification lets the first pilot proceed without resolving that separate semantic choice. [S1 addendum]

Arithmetic-only structure test: the quoted composition (i+j) mod 10 is commutative and associative. A repeated sum can solve that restricted algebra without learning operand order or tree grouping. Use it for finite behavioral checks, not as the sole evidence of expression-structure

reasoning. Confirm the actual code's contract first. [S1; mathematical derivation]

17 of 39: this is about 43.6%, not two-thirds. Preserve the exact unsupported-leader list rather than the addendum's inconsistent fraction. [S1]

Old checkpoints: a shape change does not erase the validity of an old artifact under its original code. It prevents silent loading into a changed model. Migrations require explicit identities and initialized/missing tensor reports. [S2]

Evaluation trust: the reported scorer gap needs an external trusted anchor and independent rescoring, not merely a digest that the same run can rewrite. Historical reports remain preserved. [S2]

What Fable is being asked to prove

Build one complete example in which a versioned CAS representation preserves necessary structure, a learned module uses it to produce a checked typed result, a meaningful intermediate correction changes downstream behavior as intended, and the checkpoint survives a fresh restart. Then test whether integrating that mechanism helps Helm on declared held-out tasks.

An excellent negative result is preferable to a false positive. But an absent optional provider, an unresolved unrelated namespace, or a failed first mechanism is not a reason to stop implementing the unblocked core.